Dynamic Load Balancer & Health Monitor

Computer Networks (CN) Project - Technical System Documentation

1. Executive Summary

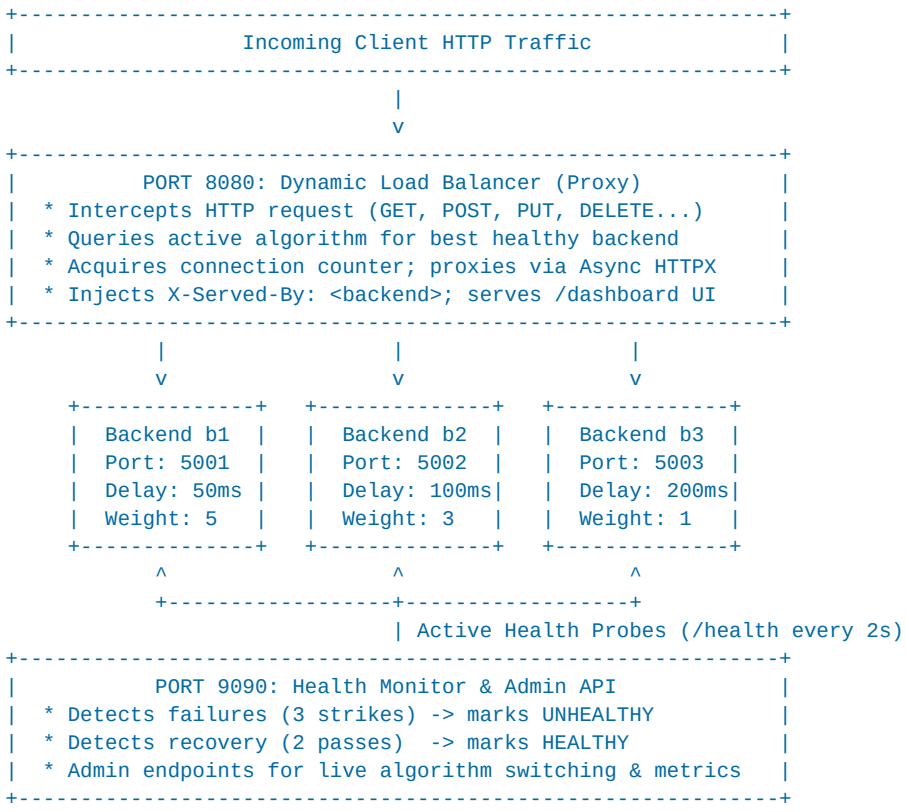
This project is a high-performance **Layer 7 HTTP Reverse Proxy and Dynamic Load Balancer** implemented in Python using FastAPI and Uvicorn. It acts as an intelligent traffic gateway that receives incoming client requests and distributes them evenly across a pool of backend servers, preventing server overload, eliminating single points of failure through active health checks, and supporting dynamic runtime reconfiguration without proxy restarts.

- **Dual-Server Architecture:** Concurrently runs the high-throughput Reverse Proxy on port 8080 and the administrative control plane on port 9090.
- **Zero-Downtime Operations:** Algorithms and backend node pools can be dynamically altered at runtime via REST Admin endpoints.
- **Active Fault Tolerance:** Background heartbeat prober identifies dead servers, isolations them within 3 strikes, and restores recovered nodes with a 10s slow-start ramp-up.
- **Built-in Monitoring & Benchmark Suite:** Real-time browser dashboard, asynchronous load generator, and statistical percentile analysis (P50, P95, P99).

2. Technology Stack

Component	Technology	Purpose
Language	Python 3.13	Asynchronous coroutines (async/await) and event loop.
Web Framework	FastAPI / Starlette	High-speed ASGI proxy routing and Admin API.
ASGI Server	Uvicorn	Runs proxy (8080) and admin (9090) concurrently.
HTTP Client	HTTPX	Non-blocking proxying, streaming, and health checks.
Configuration	PyYAML	Parses declarative backend parameters (config.yaml).
Test Suite	Pytest + pytest-asyncio	23 automated unit, failover, and integration tests.
Frontend UI	HTML5 + CSS3 + JS	Real-time glassmorphism dashboard polling Admin API.

3. System Architecture & Request Lifecycle



4. Implemented Load Balancing Algorithms

1. **Round Robin:** Distributes requests in a strict circular rotation across all healthy backends (b1 -> b2 -> b3 -> b1). Simple and deterministic.
2. **Smooth Weighted Round Robin (Nginx-style):** Distributes requests based on server weights (5:3:1) while smoothly interleaving selections to avoid bursting the heaviest server.
3. **Least Connections:** Dynamically tracks concurrent active requests per server and routes incoming traffic to the server with the lowest live load.
4. **IP Hash:** Hashes client IP addresses modulo the healthy backend count to provide session affinity (sticky sessions).

5. Active Health Monitoring & Failover

The health monitor runs as a background task concurrently with the proxy:

- **Heartbeat Interval:** Pings GET /health on all servers every 2.0s with a 1.0s timeout.
- **Fault Isolation:** After 3 consecutive failed health probes, the backend is transitioned to UNHEALTHY and excluded from traffic routing immediately.
- **Safe Recovery & Slow Start:** A recovering backend must pass 2 consecutive checks before rejoining, followed by a 10-second weight ramp-up to prevent thundering herds.

6. Step-by-Step Running Instructions

```
# 1. Start all 3 demo backends (Terminal 1):
.\.venv\Scripts\Activate.ps1
python tools/failure_sim.py demo

# 2. Start the Load Balancer & Admin Plane (Terminal 2):
.\.venv\Scripts\Activate.ps1
python -m loadbalancer.main

# 3. Test & Benchmark (Terminal 3):
curl.exe http://127.0.0.1:8080/some/path
python tools/load_gen.py --requests 100 --concurrency 10
python tools/analyze_results.py results/load_test.csv

# 4. Access Live Web Dashboard:
http://127.0.0.1:8080/dashboard

# 5. Run automated test suite:
pytest
```

7. Presentation Walkthrough for Teammates & Faculty

1. **Open Dashboard:** Navigate to http://127.0.0.1:8080/dashboard to showcase live node statuses and metrics.
2. **Send Load:** Run python tools/load_gen.py --requests 100 --concurrency 10. Point out equal request distribution under Round Robin.
3. **Switch Algorithm Live:** Use the Dashboard dropdown to switch to weighted_rr. Point out that no proxy restart was required.
4. **Demonstrate Auto-Failover:** Stop backend b2. The health monitor detects the failure, marks it red, and reroutes all traffic to b1 and b3 with 100% success rate.
5. **Prove Performance:** Show python tools/analyze_results.py results/load_test.csv displaying P50, P95, and P99 latency figures.